

# Custom Spotify Application

CS 4850-01 Spring Semester 2026

Spotify SP-27 Yellow

## Members:

Noah Cross

Ian Campbell

Spenser Catchpole

Professor: Sharon Perry 4/24/2026

## Website Link

<https://new-working-spotify-client-yellow.vercel.app/>

## Github Link

<https://github.com/zeussyt/new-working-spotify-client-yellow>

## Stats and Status:

6,381 lines of code

Components: React Native, GitHub, Vercel, Render

120 man hours

100% complete

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1.0 Introduction.....</b>                     | <b>4</b>  |
| 1.1 Project Overview.....                        | 4         |
| 1.2 Scope.....                                   | 4         |
| <b>2.0 Requirements.....</b>                     | <b>4</b>  |
| 2.1 Functional Requirements.....                 | 4         |
| 2.2 Nonfunctional Requirements.....              | 5         |
| 2.3 System Requirements.....                     | 5         |
| <b>3.0 Design.....</b>                           | <b>5</b>  |
| <b>4.0 Development.....</b>                      | <b>6</b>  |
| 4.1 Development Overview.....                    | 6         |
| 4.1.1 API Integration.....                       | 6         |
| 4.1.2 Authentication System.....                 | 6         |
| 4.1.3 Database.....                              | 6         |
| <b>4.2 Core Application Features.....</b>        | <b>7</b>  |
| 4.2.1 Top Tracks.....                            | 7         |
| 4.2.2 Playlists*.....                            | 7         |
| 4.2.3 Playlist Randomization *.....              | 7         |
| <b>4.3 Audio and Media Handling.....</b>         | <b>7</b>  |
| 4.3.1 Playback and Metadata Requests.....        | 7         |
| <b>4.4 API Constraints and Adjustments.....</b>  | <b>8</b>  |
| 4.4.1 Deprecated Features.....                   | 8         |
| <b>4.5 Frontend Development.....</b>             | <b>8</b>  |
| 4.5.1 UI Development.....                        | 8         |
| 4.5.2 Performance Optimization.....              | 8         |
| 4.5.3 Repository Diagram.....                    | 9         |
| <b>5.0 Testing Summary.....</b>                  | <b>11</b> |
| 5.1. Scope of Testing.....                       | 11        |
| 5.1.1. In scope.....                             | 11        |
| 5.1.2. Out of scope.....                         | 11        |
| 5.2. Analysis of Scope and Test Focus Areas..... | 11        |
| 5.2.1. Release Content.....                      | 11        |
| 5.3 Regression Testing.....                      | 12        |
| 5.4. Platform Testing.....                       | 12        |
| 5.5. Progression Test Objectives.....            | 12        |
| 5.6 Other Testing.....                           | 13        |
| 5.6.1. Security.....                             | 13        |
| 5.7 Stress & Volume Testing (S&V).....           | 13        |
| 5.8. Connectivity Testing (CT).....              | 13        |
| 5.9 Unit Testing.....                            | 14        |
| 5.10 Integration Testing.....                    | 14        |
| <b>6. Test Strategy.....</b>                     | <b>14</b> |

|                                                                                                                                           |           |
|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 6.1 Build strategy.....                                                                                                                   | 14        |
| 6.2 Access to other applications.....                                                                                                     | 14        |
| 6.3. Testing Requirements.....                                                                                                            | 15        |
| 6.4 Data Requirements.....                                                                                                                | 15        |
| 6.5 Resources & Skills.....                                                                                                               | 15        |
| 6.6. Testing Tools.....                                                                                                                   | 15        |
| The only tool needed for testing is a user's device and valid internet connectivity. Everything else is handled on our app's backend..... | 15        |
| <b>7. Assumptions and Dependencies.....</b>                                                                                               | <b>16</b> |
| 7.1. Assumptions.....                                                                                                                     | 16        |
| 7.2. Dependencies.....                                                                                                                    | 16        |
| <b>8.0 Summary.....</b>                                                                                                                   | <b>16</b> |

# 1.0 Introduction

## 1.1 Project Overview

Our team set out to create an effective, streamlined music streaming web service based on the Spotify API. Unneeded song suggestions do not constantly occupy the screen, and the user is only shown what is necessary when it is necessary. As a result, the UI is much easier to interact with and provides a seamless user experience, while not compromising on audio quality or song availability.

More specifically, the app allows users to login and authenticate through Spotify, create and edit playlists, and search for songs. Our main goal was to make a streamlined version of Spotify, meaning the UI is more accessible and has less information on the screen at any given time. The backend was developed using JavaScript, which made calls to the Spotify API. This means that all media controls were handled using the Spotify API. Node.js, a popular modular based design system, was used to host the backend.

## 1.2 Scope

This document describes the development of the Spotify based web application. The scope of this project includes the implementation of the streamlined user interface, backend handling of API requests, media control functions and user authentication via Spotify's OAuth 2.0. The project in its current form does not include any sort of "artificial intelligence" based functions, nor does it track any sort of user statistics. Additionally, it is important to note that the functionality of this application is entirely dependent on the continued uptime of the Spotify API. There have been times during development where the API went down, and the app could no longer function. Finally, the Spotify API only allows for certain features without payment, meaning things like database hosting cannot be done. This will be discussed later in the document.

# 2.0 Requirements

## 2.1 Functional Requirements

Login:

- User attempts to login and is sent to the Spotify login page
- Authenticate the user using Spotify's OAuth 2.0
- Provide the user with a session cookie to keep them logged in

Media Playback:

- User can play songs with the click of a button

- User can skip songs\*
- Songs can be paused at any time

Navigation:

- Switch from one screen to any other screen with a single click
- Home screen
- Search screen
- Playlist creation screen
- Library screen

Playlist Creation \*

- Create a playlist and assign it a name
- Add/Remove songs
- Change song order

Song Search

- Navigate to the search screen
- Type in a query for a band, song or album
- Display only the results that exactly match the query

## 2.2 Nonfunctional Requirements

Usability

- Anyone that has used an audio streaming service before can easily integrate into this application
- Access any screen from the user's current screen with 1 button press

Security

- Secure login through Spotify's OAuth 2.0

## 2.3 System Requirements

The user will need:

- A desktop or laptop
- Minimum 4 gigabytes of ram
- Internet connection
- Web browser
- Spotify Account (does have to be premium to use all features)

## 3.0 Design

Developing a clean and easy to use design was one of the main goals of this application. We set out to have an application that included much less “noise” on the screen than any other music streaming service. This has resulted in a streamlined UI that allows for easy navigation. Users can access any of the 4 screens from whatever current screen they are on with a single click. For example, if the user is on the home page, they can transfer to the library page with just one button and vice versa.

## 4.0 Development

### 4.1 Development Overview

#### 4.1.1 API Integration

Our application integrates with the Spotify Web API to provide user data and also media content, such as the actual audio files the user will listen to. API endpoints are used to get song metadata, like song name, albums and playlists, as well as editing user playlists and playback which is discussed in detail later in this document. These API calls are integrated into our Vercel frontend environment, making for a seamless integration and user experience.

The final webhosted version of the app runs on Vercel which effectively hosts the frontend requirements, and is closely related to React. During the early stages of development we used React, though development using React was nearly impossible to deploy without paying for its service.

Spotify's API ultimately served as a bottleneck in this application. Some features cannot be accessed without paying for the hosting services. For example, we are unable to create a playlist from the application because we are not hosting a database to store the information on. We have created the features that can be done without paying for further services.

#### 4.1.2 Authentication System

Our team utilized the existing Spotify OAuth 2.0 authentication system. OAuth 2.0 is the industry standard and is considered very secure as it utilizes temporary tokens without storing passwords. This means that users log in via Spotify, which then grants them access to our application. The application then gets tokens to make API requests, allowing users to access features on the app. This is a secure method that does not jeopardize the user's data.

To explain the authentication system in more detail, the OAuth 2.0 system authenticates via Spotify, which then gives a secure connection to the Spotify API. Users are then redirected to Spotify's authorization endpoint to log in, at which point the user receives an access token and a refresh token. The access token allows for an easier logging in process over a short time period, while the refresh token is used for new access once the access token expires. This means that no user data is stored, including their password.

#### 4.1.3 Database

The system is not currently running on a dedicated database, all database systems require payment which we are not currently provided. However, the architecture to use a database is currently present and a database can effectively be plugged into the code directly. All functions that require a database are denoted with a \* in the document.

## 4.2 Core Application Features

### 4.2.1 Top Tracks

Our application will store the user's short term listening history. This allows the user to see their top tracks for the past month from the library page. This was achieved by storing the metadata from the song and keeping track of song names and then incrementing the number of plays each time the user streams that song. The user can also see what artist they listened to the most that month.

### 4.2.2 Playlists\*

Users are able to make their own custom playlists through the Spotify Web API. Spotify has a built-in playlist endpoint. When a user goes to create a playlist, our application sends an authorized request to Spotify, which includes the user's access token and the required metadata which the user needs to enter, such as playlist name and description. Spotify then sends back playlist ID, which acts as its own entity within our custom app. The user can now add songs, remove songs and change song order.

### 4.2.3 Playlist Randomization \*

A custom randomization algorithm has been developed for our application. A simple random number generator algorithm has been developed. Spotify's shuffle algorithm is based on what you are most likely to listen to, rather than having it be purely random. For example, if you listen to 5 songs on your playlist more than the other songs, it will likely start off by playing those 5 songs. Our application makes a purely random shuffle queue.

## 4.3 Audio and Media Handling

### 4.3.1 Playback and Metadata Requests

The implementation of the audio was very straightforward. We utilize the Spotify Web API and the Spotify Playback SDK. This means that every media control, such as starting a song, pausing a song, skipping \*, etc. is handled using authorized requests to the Spotify Playback SDK. This allows for the best user experience, with the highest-quality audio possible. This is also when song metadata is retrieved. The application will make requests to the Spotify Web API endpoints, and retrieve song title, duration, artist, album art, etc.

## 4.4 API Constraints and Adjustments

### 4.4.1 Deprecated Features

During the brainstorming and early implementation phases of the project, our team wanted to provide detailed information to the user about the music they were listening to. Specifically, we wanted song analytics to be displayed, such as the song's BPM, genre and key. This would have given our application more accessibility to users who are musically inclined, while not being distracting for users who are not. Ultimately, we ran into several problems implementing this feature. It would have required users to have a premium Spotify account, it would have added complexity for SDK calls and it would have made permissions needlessly complicated. As a result, we decided to end implementation of this feature.

## 4.5 Frontend Development

### 4.5.1 UI Development

The user interface was developed initially using React, which is a popular modular based design system. Ultimately we changed to [node.js](#) through Vercel as React required payment to fully integrate. The layout and navigation structure underwent several changes throughout the development process. For example, we initially had certain pages that could only be accessed through several button presses. We decided this was a poor design choice and made all pages accessible from the home page. The layout was also streamlined, such as having fewer pages and instead only using a home and library page. As a result, usability is significantly improved for users, and maintenance is less complicated for our developers. Vercel is used to host the front end.

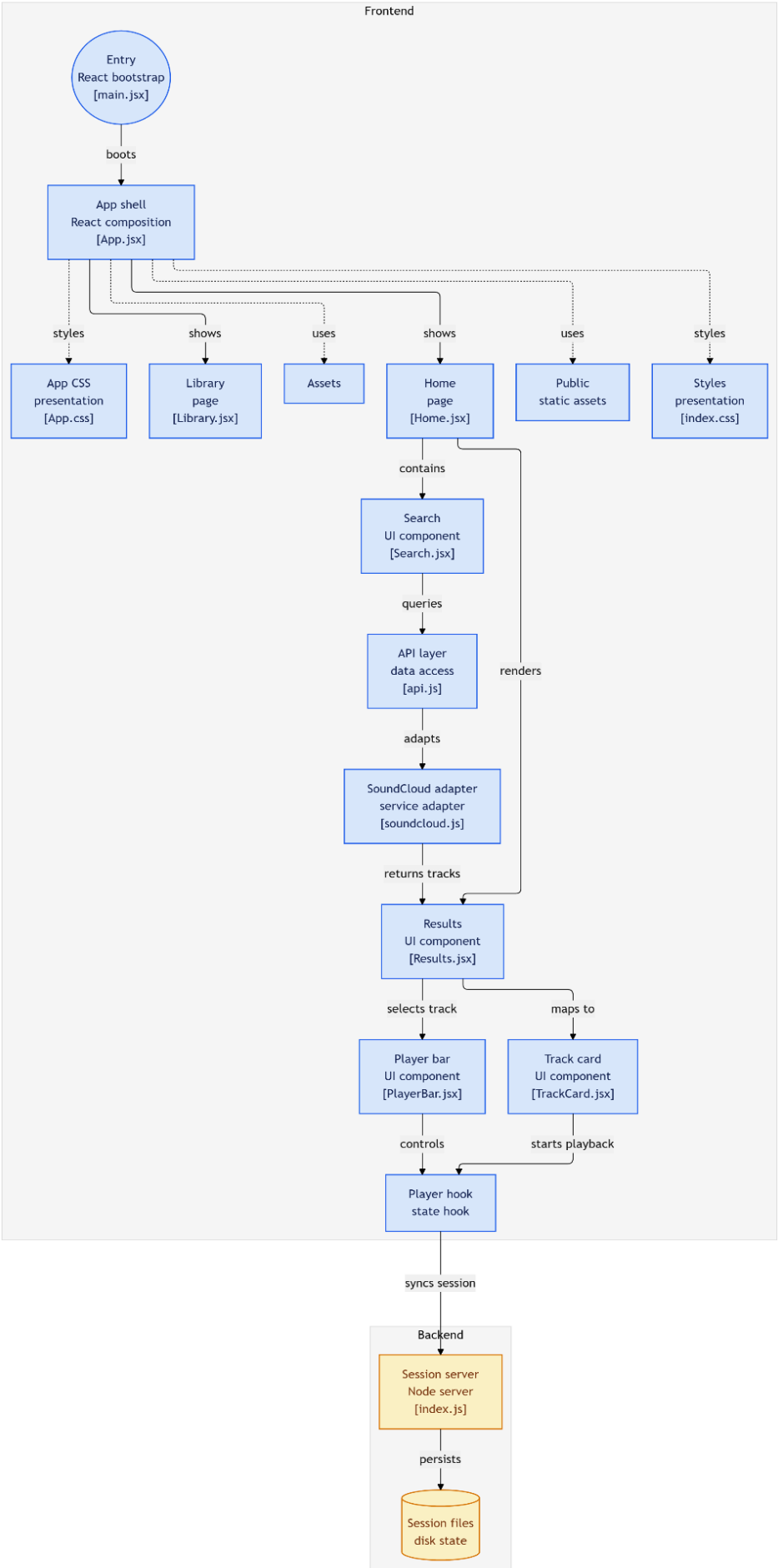
### 4.5.2 Performance Optimization

Performance optimization was one of our main concerns in initial development, but ultimately there were very few issues in this regard. Repeated API requests were reduced using caching metadata when possible. [Node.js](#) and Vercel are also very efficient for rendering album covers and other images. For example, if an album cover stays the same from one song to the next, it stores the image data in order to not unnecessarily re-render the image.



### 4.5.3 Repository Diagram

```
...
new-working-spotify-client-yellow/
├── .gitignore
├── README.md
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── public/
│   ├── ian_project.jpg
│   ├── ian_project.png
│   ├── noah_project.jpg
│   ├── noah_project.png
│   ├── spenser_project.jpg
│   ├── spenser_project.png
│   └── vite.svg
├── server/
│   ├── .gitignore
│   ├── index.js
│   ├── package-lock.json
│   ├── package.json
│   └── sessions/
│       ├── ApxeYNsrS3LMrvzeiqo-jjhw0fGqvnYz.json
│       ├── enu5Z1tybadAAaCP4B4tqFYVPigP3Y96.json
│       └── vEq9-xUhCtUuJjcAXBtWEphnQVJxnP3.json
├── src/
│   ├── App.css
│   ├── App.jsx
│   ├── api/
│   │   ├── api.js
│   │   └── soundcloud.js
│   ├── assets/
│   │   └── react.svg
│   ├── components/
│   │   ├── PlayerBar.jsx
│   │   ├── Results.jsx
│   │   ├── Search.jsx
│   │   └── TrackCard.jsx
│   ├── hooks/
│   │   └── useSpotifyPlayer.js
│   ├── index.css
│   ├── main.jsx
│   └── pages/
│       ├── Home.jsx
│       └── Library.jsx
└── vite.config.js
...
```



## 5.0 Testing Summary

### 5.1. Scope of Testing

#### 5.1.1. In scope

Our scope of testing for this document is the ability to login through our application, and the ability to use all functionalities directly implemented by our developers. These functionalities include:

- Login “flow” (OAuth call, token handling)
- Playing songs from the application
- Creating playlists\*
- Editing playlists (adding and removing songs)\*
- Searching for songs
- Music playback and controls
- UI interaction and navigation

#### 5.1.2. Out of scope

There are interactions that will be used with our application that we will not be testing, as we do not have direct control over them. Our app uses a lot of API calls to Spotify, meaning certain functions are dependent on Spotify’s backend to be working. Things outside of the scope of this testing process include:

- Spotify’s login authentication system
- Spotify’s backend and API availability
- User network reliability
- Connection to database

## 5.2. Analysis of Scope and Test Focus Areas

### 5.2.1. Release Content

The current release includes all features of the Spotify based application that are fully implemented and functional. These include:

- User authentication and login through Spotify API
- Song search functionality
- Music playback and controls
- Playlist creation and editing\*
- UI interaction and navigation

## 5.3 Regression Testing

Regression testing ensures that all features implemented in earlier versions of the application are still functional in later versions of the app. This is useful mostly because the app was programmed modularly, one feature was created and refined before working on another feature. Regression testing will make sure that no old functionalities were accidentally deprecated.

## 5.4. Platform Testing

Our initial experiments were performed on a Windows operating system, with 16 gigabytes of ram and an Intel processor. Computation concerns are minimal, any machine running on 8 gigabytes of ram and a gpu and cpu made within the last 15 years will run this application without difficulty. MacOS and Linux will also work.

## 5.5. Progression Test Objectives

Progression test objectives that will be covered:

| Ref                   | Function                    | Test Objective                                             | Evaluation Criteria                                                                                                     | P      |
|-----------------------|-----------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------|
| Function to be tested |                             |                                                            |                                                                                                                         |        |
| SP-Login              | Login Flow                  | Verify that the user can login in via Spotify              | User is authenticated and redirected to the home screen                                                                 | High   |
| SP-Search             | Search                      | Verify that users can enter a search query and see results | The user gets results within 5 seconds of entering a query, and the results have the same name as the query.            | High   |
| SP-Playback           | Music Playback and Controls | Verify that users can pause, skip and replay songs         | Each button responds quickly and has the expected result. For example, hitting the pause button stops the song within 1 | Medium |

|                   |                               |                                                                              |                                                                                                                                                              |        |
|-------------------|-------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                   |                               |                                                                              | second of the button press.                                                                                                                                  |        |
| SP-Playlist       | Playlist Creation and Editing | Verify users can create a playlist and edit it by adding and removing songs. | Hitting the "create playlist" button creates a popup allowing the user to name the playlist. 5 songs will be added, and they need to appear within 1 minute. | Low    |
| SP-User Interface | UI Interaction and Navigation | Verify that users can navigate through the app quickly                       | All pages will be cycled through and need to appear within 2 seconds. The UI looks as it should with no unintentional buttons or wording on the screen.      | Medium |

## 5.6 Other Testing

### 5.6.1. Security

Security testing is minimal as potential security vulnerabilities are virtually impossible. No sensitive information is passed to our application directly, as Spotify itself handles all login information and uses OAuth 2.0.

## 5.7 Stress & Volume Testing (S&V)

A Load test focuses on evaluating how this prototype performs under heavy data and interaction loads, ensuring the UI and the API remains responsive and stable. We will test the prototype with the following:

- Increasing dataset size to 1,000 tracks
- Repeatedly generate playlists
- Simulate rapid user interactions
- Monitor the FPS and memory usage

## 5.8. Connectivity Testing (CT)

Connectivity testing is a process that allows developers to verify whether data is correctly being processed through an application or between endpoints. For our app, we will test the following:

- App to Spotify API calls locally (and webhosted eventually)
- User authentication and validation through OAuth2.0
- Proper data retrieval from Spotify (this includes playlist data, song data, artist data, etc.)

## 5.9 Unit Testing

Unit testing is relatively straightforward with our app. For the frontend, we will compare our javascript (.jsx) files of each tab to the formatting of the actual in-app tab and make sure it looks correct and presentable. For the backend, there are a few ways we will test each function. First, we make sure the backend is connected to the frontend via print statements on the backend. Once we verify that the two are connected, we will navigate to each tab and test the functionality of each backend call (ie. search, makePlaylist, etc). After a function is working, and if it is needed, we will print a statement to make sure that said function is properly obtaining information from the Spotify API.

## 5.10 Integration Testing

For integration testing, we will test each tab / page of our app by making sure that nothing breaks while using multiple functions at once. An example of this would be making sure that a playlist brings up the right songs even though a search is being queried in said playlist.

# 6. Test Strategy

## 6.1 Build strategy

Our build strategy for this app is simple. At the base level, we value functionality over aesthetics. For every function of the app, we make sure it works and is as polished as possible before moving on to the next function. To accomplish this our team uses a shared GitHub repository where each member has access to the latest version. Every new feature or fix is tested locally then reviewed before merging into the main branch. This strategy ensures that the prototype remains stable, easy to run, and ready for any testing throughout the project.

## 6.2 Access to other applications

As long as the user has access to all files within our project repo OR our webpage, there should be no need for external applications.

## 6.3. Testing Requirements

Each person involved in testing will need the following access:

- A web browser with access to the intranet OR smartphone with Appstore / PlayStore access
- Access to a valid Spotify Premium Account

## 6.4 Data Requirements

Data required for testing is:

- Access to the user's Spotify Account

## 6.5 Resources & Skills

For testing, users need to know:

- General understanding of workflow and web application navigation
- Knowledge of OAuth 2.0 functionality
- Be familiar with JavaScript, [node.js](https://nodejs.org/), Vercel and Render

## 6.6. Testing Tools

The only tool needed for testing is a user's device and valid internet connectivity. Everything else is handled on our app's backend.

For example:

The following tools will be used for testing, mostly for recording results:

| Process              | Tool            |
|----------------------|-----------------|
| Test case creation   | Microsoft Word  |
| Test case tracking   | Microsoft Excel |
| Test case execution  | Manual          |
| Test case management | Microsoft Excel |
| Defect management    | Microsoft Excel |

## 7. Assumptions and Dependencies

### 7.1. Assumptions

These are the assumptions that have been made for executing the current application:

- The Spotify Web API is functional
- User has a Spotify account
- User has a premium account for certain functions
- All files have been properly configured

### 7.2. Dependencies

All dependencies for running the application are listed here:

- Internet connection
- Backend and frontend files running
- Spotify's authorization system is functional
- A database needs to be connected to the system in order for some functions

## 8.0 Summary

This paper shows and demonstrates the development of a full-stack application that was created using the Spotify API. All major objectives have been met, as users can safely authenticate into the application through Spotify's login page, search for music, play music at will and create and edit playlists. All core features were implemented as we intended, considering the restraints of the Spotify API.

Throughout the development process, key concepts and functions have been implemented, such as API integration, user authentication, as well as frontend and backend development. Though many challenges were encountered, especially with the OAuth 2.0 system, ultimately the problems were addressed and resolved.

Small changes will continue to be made to the application, particularly small UI tweaks, but more importantly, the application will continue to be monitored so as to maintain a constant uptime. However, the application is still fully functional and will streamline the use of music streaming services.

**Sections denoted with a \* are functions that are dependent on a database being connected to the system.**